redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

Team Name : Catalyst Playmakers

Team Leader Name : Mohammad Aazen

Problem Statement : Intelligent Candidate Discovery & Ranking

Solution Overview

- **A structural-reasoning ranker.** It reads each profile the way a recruiter would — judging what a candidate actually did, not which keywords they listed.
- **Seven explainable components** combine into one fit score, then a behavioral-availability modifier adjusts for who is actually reachable.
- **Title + career evidence outweigh the skills list** — so a Marketing Manager stuffed with AI keywords cannot rank, no matter how perfect the skill list looks.
- **Consistency checks neutralise honeypots** and impossible profiles automatically, with no special-casing.
- **Every candidate gets an honest, fact-grounded reason** — no black box and no hallucinated skills.
- **Production-realistic.** CPU-only, no network at ranking time, ~45s for the full 100,000-candidate pool.

JD Understanding & Candidate Evaluation

- **Extracted must-haves:** production embeddings / retrieval, vector-search infra, ranking & recsys, rigorous evaluation (NDCG / MRR / MAP), strong production Python; 5–9 yrs at product (not services) companies.
- **Encoded disqualifiers:** research-only with no production, consulting-only career, title-chasing, no recent code, and CV / speech / robotics without NLP / IR.
- **Most important signals:** actual job titles and career-history descriptions, skill trust (proficiency × real usage × endorsements × assessment), and behavioral availability.
- **Beyond keywords:** evidence is weighted by WHERE it appears — proof inside real work history counts ~3× a bare skills-list keyword.
- **Finds the hidden fits:** plain-language candidates who built a recommender or search system without the buzzwords still surface.

Ranking Methodology

- **Retrieve:** stream all 100k profiles once in constant memory and score every candidate — no lossy keyword pre-filter that could drop a strong fit.
- **Score — seven 0–1 components:** role relevance 0.26, core requirements 0.24, experience fit 0.12, semantic 0.12, skill trust 0.10, trajectory 0.10, location 0.06.
- **Combine:** weighted sum → ideal-profile bonus to sharpen the top 10 → × behavioral availability modifier (0.55–1.12).
- **Honeypot consistency tax** collapses internally-impossible profiles so they can never reach the top ranks.
- **Heuristic and fully deterministic:** no per-candidate model inference — fast, reproducible, and explainable, exactly as the compute rules demand.

Explainability & Data Validation

- **Reasons are generated from the scoring facts:** each explanation is built from the exact evidence used to score that candidate — title, years, requirement evidence, flags, availability.
- **Hallucination-proof by construction:** reasoning can only cite evidence already extracted from the profile; it cannot invent a skill the candidate does not have.
- **Suspicious profiles are flagged:** tenure longer than the whole career, expert skills with 0 months used, and dates that don't add up trigger a consistency penalty that collapses the score.
- **Generalises beyond the ~80 known honeypots** — verified 0 honeypots and 0 keyword-stuffers in the top 100; 0 honeypots in the top 10.
- **Validated:** 100/100 unique reasoning strings, and the output passes the official validate_submission.py.

End-to-End Workflow

- **1 · Offline (once):** an LLM parses the job description into a structured rubric → committed jd_rubric.json. Network allowed here; done a single time.
- **2 · Load rubric:** rank.py reads the committed rubric — from here on there is zero network.
- **3 · Stream candidates:** read candidates.jsonl line by line and extract features for each profile.
- **4 · Score:** honeypot consistency check + seven-component scoring + behavioral modifier.
- **5 · Rank:** maintain a running top-K heap, sort, normalise scores, and break ties by candidate_id.
- **6 · Output:** write submission.csv — the top 100 with rank, score, and a per-candidate reason.

System Architecture

- **OFFLINE · pre-computation (network OK):** build_rubric.py → OpenRouter → artifacts/jd_rubric.json (committed). One call; no candidate data is ever sent to an LLM.
- **RANKING · rank.py (the judged step):** pure-Python standard library, CPU-only, no network. Streams candidates → scorer → top-100 CSV.
- **ranker/honeypot.py:** internal-consistency / impossible-profile detection.
- **ranker/scoring.py:** the seven components + ideal-profile bonus + behavioral modifier.
- **ranker/reasoning.py:** deterministic, fact-grounded explanations.
- **Compute envelope:** < 16 GB RAM, < 5 min (≈ 45s), no GPU — reproducible in a no-network Docker sandbox.

Results & Performance

- **0 keyword-stuffers in the top 100** — off-domain titles with padded AI skill lists are excluded.
- **0 honeypots in the top 100** (well under the 10% disqualification threshold) and 0 in the top 10.
- **Clean shortlist:** the top 100 is entirely relevant ML / AI / Search / NLP / RecSys / Data-Science roles.
- **Trustworthy explanations:** 100/100 unique, fact-grounded reasons; passes the official format validator.
- **Meets the constraints:** ≈ 45s for all 100,000 candidates — CPU-only, no network, < 16 GB RAM — inside the 5-minute budget.
- **Reproduction-guarded:** a no-network test suite checks the output contract before every run.

Technologies Used

- **Python 3 standard library (ranking step):** zero heavy dependencies → trivially reproducible, no model downloads, no network.
- **JSON rubric as the single config:** weights, concept-term lists, and disqualifier markers live in one tunable file.
- **OpenRouter (offline, optional):** an LLM parses the JD into the rubric; the output is committed, so the ranker needs no key at run time.
- **Streamlit:** the hosted sandbox demo (app.py) for the required reproducibility check.
- **Why this stack:** the rules forbid GPUs, hosted-LLM calls, and network during ranking — so the intelligence lives in engineered heuristics + an offline-derived rubric, not runtime inference.

Submission Assets

- **GitHub repo:** rank.py + ranker/ modules, committed rubric, test suite, README, and reproduce.sh.
- **Ranked output:** submission.csv — the top 100, format-validated.
- **Deck (PDF):** this presentation — approach, architecture, and results.
- **Sandbox:** the Streamlit app (app.py) on HuggingFace Spaces / Streamlit Cloud.
- **Reproduce command:** python rank.py –candidates ./candidates.jsonl –out ./submission.csv

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU